



Documentación de Ciberseguridad

Nombre Fantasía, de la nueva empresa: S.M. NOVA

Grupo: 3° MM

Turno: Vespertino

Unidad Curricular: BT

Nombres de los integrantes del grupo: Arboleda Ihara
Perazzo Dorian

Fecha de entrega: 14/09/2026

Instituto Tecnológico Superior Arias Balparda
Gral. Flores 3591 esq. Bvar. José Batlle y Ordoñez – Montevideo



ANEP



UTU

DIRECCIÓN GENERAL
DE EDUCACIÓN
TÉCNICO PROFESIONAL



Instituto Tecnológico Superior
UTU

Participantes

Listado alfabético por apellido de todos los colaboradores del documento, indicando el solicitante:

- Arboleda, Ihara — redactor de este documento.
- Perazzo, Dorian — colaborador de este documento.
- Solicitante del documento: I.T.S. Arias – Balparda.



Table of Contents

Participantes.....	3
1. Introducción y alcance.....	
2. Amenazas identificadas.....	4
2.1 Amenaza evaluada y no priorizada: Ransomware.....	6
3. Vulnerabilidades de la infraestructura y cómo las vamos a mitigar.....	7
3.1 Servidor.....	7
3.2 Base de datos.....	8
3.3 Red interna.....	9
4. Control de acceso y roles de usuarios.....	10
4.1 Usuarios de infraestructura del servidor.....	11
5. Política de contraseñas.....	12
6. Políticas y medidas de seguridad generales.....	14
7. Implementaciones de seguridad en el servidor.....	16
7.1. Fail2ban.....	16
7.1.1. ¿Qué es?.....	17
7.1.2. Instalación.....	17
7.2 OSSEC.....	18
7.2.1. ¿Qué es?.....	18
7.2.2. Instalación.....	19
7.3 Grafana.....	20
7.3.1 ¿Qué es?.....	20
7.3.2 Instalación.....	20
8. Esquema de red.....	22
9. Encriptación de contraseñas.....	22
10. Bibliografía.....	23

Participantes

Listado alfabético por apellido de todos los colaboradores del documento, indicando el solicitante:

- Arboleda, Ihara – redactora de este documento.



1. Introducción y alcance

Este documento define el alcance de la primera entrega de nuestro grupo en la unidad curricular Ciberseguridad, dentro del proyecto Canopus (nuestro Sistema de Gestión Deportiva Modular, SGDM), conforme a lo acordado por el equipo en la reunión del 15 de julio de 2026 (Acta N.º 5).

En esta entrega relevamos y analizamos las amenazas específicas que pueden afectar a Canopus, identificamos las vulnerabilidades concretas de los tres componentes de nuestra infraestructura (servidor, base de datos y red interna), definimos el modelo de control de acceso y roles de usuario, la política de contraseñas que vamos a aplicar, y las políticas y medidas de seguridad generales que se desprenden de todo el análisis.

Decidimos no quedarnos únicamente en identificar los riesgos: en cada sección explicamos también cómo pensamos resolver cada problema dentro de nuestra propia implementación, que va a estar construida con PHP, MySQL, Apache y Docker, siguiendo la arquitectura MVC definida en la documentación funcional del proyecto. Nos parece que un documento de seguridad que solo enumera amenazas sin decir cómo se solucionan no le sirve de mucho a nuestro propio equipo de desarrollo, así que priorizamos que cada punto quede accionable.

2. Amenazas identificadas

Relevamos las siguientes amenazas específicas para nuestro sistema Canopus. En vez de describirlas de forma genérica, explicamos en cada caso cómo afectarían concretamente a nuestra plataforma —sus módulos, su base de datos y sus roles— y qué medida puntual vamos a aplicar para mitigarla.

**ANEP****UTU**DIRECCIÓN GENERAL
DE EDUCACIÓN
TÉCNICO PROFESIONAL

Instituto Tecnológico Superior

UTU

Amenaza	Descripción y cómo la vamos a mitigar
Malware	Software malicioso que podría infectar el servidor donde corren Apache y PHP, comprometiendo la aplicación completa. En nuestro sistema, el punto de entrada más probable es el módulo de gestión de participantes y equipos, donde se suben fotos de perfil y logos: si solo validamos el archivo por su extensión y no por su contenido real, alguien podría subir un script disfrazado de imagen. Para evitarlo, vamos a verificar el tipo real del archivo (no solo la extensión), limitar tamaño y formatos permitidos, correr los contenedores con un usuario sin privilegios y mantener actualizadas las imágenes de Docker y las dependencias de PHP.
Phishing	Alguien podría hacerse pasar por un administrador del SGDM y pedirle a un organizador de torneo que reingrese su contraseña en un enlace falso, para robarle las credenciales y después alterar resultados o configuraciones. Esta amenaza se ve favorecida si no hay autenticación multifactor ni un canal oficial reconocible. Vamos a exigir MFA en las cuentas de Administrador general y Organizador de torneo, y vamos a dejar por escrito en el manual de usuario que el sistema nunca va a pedir la contraseña por correo.
Ataques a contraseñas	Intentos de descubrir una contraseña válida por fuerza bruta o probando credenciales filtradas de otros sitios (credential stuffing) contra el formulario de login del panel de administrador u organizador, que son las cuentas más atractivas por su capacidad de modificar resultados. Vamos a mitigarlo bloqueando temporalmente la cuenta tras varios intentos fallidos, exigiendo contraseñas largas según la política de la sección 5, y guardando siempre las contraseñas como hash.
Denegación de servicio (DoS / DDoS)	Busca saturar el servidor con muchas solicitudes para dejar la plataforma inaccesible. La vista pública de Canopus (calendarios, resultados y posiciones) es la más expuesta porque no requiere autenticación y puede recibir muchas consultas simultáneas cuando se publican resultados importantes. Vamos a aplicar límites de solicitudes (rate limiting) en las rutas públicas, cachear la información que no cambia todo el tiempo y monitorear el tráfico para detectar picos anómalos.
Explotación de vulnerabilidades (inyección SQL y XSS)	Ocurre cuando alguien aprovecha errores de programación en el código PHP para acceder o modificar datos sin autorización. En nuestros formularios de inscripción, carga de resultados y nombre de equipo se escribe directamente en MySQL; si armamos las consultas concatenando texto en vez de usar sentencias preparadas, se podría inyectar SQL, y si mostramos esos datos después sin escapar el HTML, también se podría inyectar XSS. Por eso vamos a usar exclusivamente sentencias preparadas (PDO) para todo acceso a la base y vamos a validar y escapar toda entrada y salida de datos, siguiendo el

**ANEP****UTU**DIRECCIÓN GENERAL
DE EDUCACIÓN
TÉCNICO PROFESIONALInstituto Tecnológico Superior
UTU

Amenaza	Descripción y cómo la vamos a mitigar
	OWASP Top 10.
MITM (hombre en el medio)	Consiste en interceptar la comunicación entre el navegador y el servidor para leer o modificar datos en tránsito. Sería especialmente riesgoso si alguien accede a Canopus sin HTTPS desde la red wifi del instituto o desde una red pública durante una demostración. Vamos a forzar HTTPS con certificado SSL/TLS, habilitar HSTS y evitar usar redes públicas no confiables para tareas administrativas del sistema.
Amenaza interna	Proviene de alguien con acceso legítimo que causa daño por error o a propósito: un organizador podría modificar un resultado que no le corresponde, o un participante podría intentar acceder a funciones administrativas si solo ocultamos esas opciones en la interfaz sin validar el rol en el backend. Por eso vamos a verificar siempre el rol y el torneo asignado en cada punto del servidor, vamos a registrar en auditoría cada cambio de resultado y vamos a revisar los permisos periódicamente.
Ingeniería social	Se basa en manipular a una persona para que entregue información o acceso, más que en explotar una falla técnica. Alguien podría contactar a un organizador simulando ser un participante con un pedido urgente de cambio de resultado, apostando a que la urgencia lo haga saltarse la verificación. Vamos a definir un único procedimiento obligatorio para confirmar identidad antes de cambios sensibles, y vamos a aplicar el principio de mínimo privilegio para limitar el impacto de este tipo de engaño.

2.1 Amenaza evaluada y no priorizada: Ransomware

En una primera versión de este relevamiento habíamos incluido el Ransomware como amenaza específica, pero al revisarlo con el profesor concluimos que no lo íbamos a priorizar como amenaza diferencial para Canopus, y preferimos dejar constancia de por qué en vez de sacarlo sin más. El ransomware apunta a sistemas que manejan pagos, datos de alto valor de reventa o servicios cuya caída genera pérdidas económicas directas que justifiquen pagar un rescate; Canopus, en cambio, no procesa pagos y su público objetivo son clubes, instituciones educativas y organizadores sin fines de lucro, lo que lo vuelve un blanco poco atractivo para ese modelo de extorsión. Además, el vector de entrada que necesitaría un ransomware para llegar al servidor ya está cubierto por las Tecnologías de la Información – I.T.S. Arias-Balparda – Ciberseguridad



dos primeras amenazas de esta tabla, así que tratarlo aparte hubiera duplicado la misma mitigación sin agregar un riesgo realmente distinto. De todas formas, mantenemos los respaldos automatizados y guardados fuera del servidor de producción como medida general de continuidad, porque también nos protegen ante cualquier pérdida o cifrado accidental de los datos, no solo ante un ataque de extorsión.

3. Vulnerabilidades de la infraestructura y cómo las vamos a mitigar

A diferencia de las amenazas (los peligros externos que podrían afectar al sistema), las vulnerabilidades son las debilidades concretas de nuestra infraestructura que esas amenazas podrían aprovechar. Relevamos vulnerabilidades en los tres componentes: servidor, base de datos y red interna, y para cada componente dejamos definido qué vamos a implementar para cerrarlas.

3.1 Servidor

Vulnerabilidad	Descripción
Software desactualizado	Apache, PHP, MySQL o Docker sin actualizaciones conservan fallas conocidas y ya corregidas, explotables con herramientas públicas.
Privilegios excesivos en servicios	Si un servicio corre con más permisos de los necesarios (por ejemplo como root dentro del contenedor), un atacante que lo comprometa hereda esos privilegios.
Puertos innecesarios expuestos	Servicios o puertos que Canopus no usa amplían innecesariamente la superficie de ataque.
Configuración por defecto sin ajustar	Contraseñas, usuarios o parámetros de fábrica sin modificar son un blanco fácil por ser públicos o predecibles.
Falta de límites de recursos en contenedores	Contenedores Docker sin límites de CPU o memoria permiten que un fallo o un ataque agoten los recursos del servidor físico.



Cómo lo vamos a resolver: para el software desactualizado, vamos a fijar un día fijo antes de cada entrega en el que actualizamos las imágenes base de Docker y los paquetes de Apache, PHP y MySQL, en vez de dejarlo librado a que alguien se acuerde; lo vamos a anotar como tarea recurrente en nuestro seguimiento de planificación (UTULAB). Para los privilegios excesivos, nuestros contenedores van a correr con un usuario propio sin privilegios (nunca root dentro del contenedor), definido en el Dockerfile con la instrucción USER. Para los puertos innecesarios, en el docker-compose solo vamos a exponer el puerto 443 (HTTPS) hacia afuera; el resto de los puertos internos (MySQL, por ejemplo) nos quedarán visibles únicamente entre contenedores, dentro de la red interna de Docker. Para la configuración por defecto, antes de dar por cerrada la instalación vamos a revisar y cambiar cualquier usuario, contraseña o parámetro que venga de fábrica en las imágenes que usemos, dejándolo como paso obligatorio de nuestro check de despliegue. Y para los límites de recursos, vamos a definir toques de CPU y memoria por contenedor en el docker-compose (deploy.resources.limits), para que un pico de tráfico en un torneo no deje sin recursos al resto de los servicios del servidor.

3.2 Base de datos

Vulnerabilidad	Descripción
Inyección SQL	Consultas armadas concatenando texto en vez de usar parámetros preparados permiten manipular la base desde un formulario, como el de carga de resultados.
Base de datos expuesta a la red	Si el puerto de MySQL es accesible desde fuera del servidor, cualquier equipo de esa red podría intentar conectarse directamente.
Credenciales débiles o por defecto	Usuarios de la base con contraseñas débiles o de instalación facilitan el acceso no autorizado a toda la información.
Falta de cifrado de datos sensibles	Datos como las contraseñas, guardados sin hash, quedarían totalmente expuestos ante una filtración.



Cómo lo vamos a resolver: para la inyección SQL, todo acceso a MySQL desde PHP se va a hacer consultas preparadas (PDO), nunca concatenando texto directamente en la consulta; esto lo vamos a aplicar en todos los formularios que escriben en la base, no solo en el de login, incluyendo carga de resultados, inscripción de participantes y creación de torneos. Para la base de datos expuesta a la red, el puerto de MySQL va a quedar accesible únicamente dentro de la red interna de Docker, así que ningún equipo fuera del propio stack de contenedores va a poder conectarse directo a la base. Para las credenciales débiles, vamos a crear un usuario de base de datos propio de la aplicación, distinto del usuario root de MySQL, con permisos limitados exclusivamente a las tablas de Canopus (nada de otorgarle privilegios globales), y esa contraseña va a quedar definida como variable de entorno del contenedor, no escrita en el código. Y para el cifrado de datos sensibles, las contraseñas de los usuarios del sistema se van a guardar siempre con hash mediante `password_hash()` de PHP (bcrypt), de modo que ni siquiera nosotros, con acceso directo a la base, podamos ver la contraseña original de un usuario.

3.3 Red interna

Vulnerabilidad	Descripción
Tráfico sin cifrar	La comunicación sin HTTPS queda expuesta a interceptación (MITM), permitiendo capturar credenciales o alterar datos en tránsito.
Ausencia de reglas de firewall	Sin reglas que limiten puertos y orígenes, cualquier equipo puede intentar acceder a servicios que deberían estar restringidos.
Falta de segmentación de red	Si la VM de Canopus comparte la red interna del instituto sin aislamiento adicional, un dispositivo comprometido en esa red podría llegar directo al servidor.
Sin sistema de detección de intrusos	La ausencia de un IDS/IPS retrasa la detección de tráfico anómalo o intentos de intrusión.



Cómo lo vamos a resolver: vamos a forzar HTTPS con certificado SSL en el servidor Apache, vamos a definir reglas de firewall que solo dejen pasar los puertos que la aplicación realmente usa, y vamos a evaluar incorporar fail2ban para bloquear IPs con comportamiento sospechoso en el login. En esta primera entrega dejamos identificada también la falta de un IDS/IPS como algo a resolver en una etapa posterior del proyecto, priorizando primero el cifrado del tráfico y el firewall. Sobre la falta de segmentación de red en particular: la red física y las VLAN del instituto las define la UTU, no nosotros (la misma limitación de alcance que ya dejamos aclarada en el Estudio de sistemas operativos cliente y servidor), así que nuestra capacidad de acción real está en la capa de aplicación. Dentro de ese margen, vamos a separar los contenedores Docker en redes internas distintas: una red para el contenedor de Apache/PHP y otra red interna aparte para MySQL, de modo que la base de datos solo sea alcanzable a través de la aplicación y nunca directamente desde otro equipo de la red del instituto. Esto no reemplaza una segmentación a nivel de red física, pero reduce la superficie de ataque en lo que sí podemos controlar como equipo de desarrollo. Además, vamos a dejar solicitado formalmente a la institución que la VM de Canopus se ubique en una VLAN separada del resto de la red interna, documentando ese pedido como una limitación que excede nuestro alcance directo.

4. Control de acceso y roles de usuarios

El control de acceso de Canopus lo estructuramos en cuatro etapas:

- **Identificación:** cada usuario queda registrado de forma única en el sistema (usuario o correo electrónico único).
- **Autenticación:** verificamos la identidad del usuario mediante sus credenciales, según la política de contraseñas de la sección 5.
- **Autorización:** una vez autenticado, el sistema determina qué acciones puede realizar el usuario según su rol.



- Logs: registramos en auditoría las acciones relevantes realizadas por cada usuario.

Estas etapas se aplican de forma diferenciada según el rol, definidos en la documentación funcional del proyecto:

Rol	Autenticación	Nivel de acceso	Auditoría (logs)
Administrador general	Obligatoria	Control total: usuarios, torneos, módulos, resultados, configuración general.	Registro exhaustivo de todas sus acciones, por ser el rol de mayor privilegio.
Organizador de torneo	Obligatoria	Limitado a los torneos que administra: inscripciones, rondas, resultados de esos torneos.	Registro de altas, bajas y modificaciones sobre sus torneos.
Participante	Obligatoria	Solo lectura de sus torneos, calendario, resultados y posiciones; edición de datos básicos de su perfil.	Registro de inicios de sesión y cambios de perfil.
Usuario público	No requiere	Solo lectura de información pública (torneos, calendarios, resultados, posiciones).	Registro de accesos, útil para detectar patrones de abuso o DDoS.

A nivel de implementación, en el backend vamos a manejar la sesión del usuario autenticado con las sesiones nativas de PHP, y en cada controlador vamos a validar el rol y, cuando corresponda, que el organizador sea efectivamente dueño del torneo que quiere modificar. No vamos a delegar ningún control de permisos solo a la vista: si una opción no debería estar disponible para un rol, además de ocultarla en el frontend, el backend va a rechazar la acción aunque alguien intente forzarla.

4.1 Usuarios de infraestructura del servidor

Los roles de la tabla anterior son los roles de la aplicación (quién entra a Canopus desde el navegador), pero el principio de mínimo privilegio también lo aplicamos un nivel más abajo, en el propio sistema operativo del servidor. Estos usuarios de infraestructura están detallados en el



Estudio de sistemas operativos cliente y servidor, y los resumimos acá porque son parte directa de nuestro modelo de control de acceso general:

- **root**: superusuario del sistema Linux. Lo dejamos reservado para tareas puntuales que realmente necesiten privilegios totales; no lo usamos para el trabajo diario ni para iniciar sesión de forma habitual.
- **admin-canopus**: cuenta administrativa con sudo que usa el equipo para el mantenimiento del servidor. Cada integrante tiene su propia cuenta individual dentro de este grupo (no compartimos una sola cuenta), para que quede registro de quién hizo cada cambio.
- **apache (www-data)**: cuenta de servicio con la que corre Apache. Solo tiene acceso a las carpetas de la aplicación web, sin shell de login, así que no se puede usar para iniciar sesión directamente.
- **mysql**: cuenta de servicio con la que corre el motor de base de datos, con acceso únicamente a sus propios archivos y procesos, tampoco con shell de login.
- **deploy-canopus (grupo docker)**: cuenta separada para gestionar los contenedores, sin permisos de sudo generales sobre el resto del servidor. La aislamos porque pertenecer al grupo docker equivale en la práctica a tener privilegios de root, así que no queremos que esa cuenta se use para nada más.

Con esta separación, si un atacante llegara a comprometer el proceso de Apache (por ejemplo por una vulnerabilidad en el código PHP), quedaría limitado a los permisos de la cuenta apache y no tendría automáticamente acceso a la base de datos, a los contenedores ni al resto del sistema

5. Política de contraseñas

La contraseña sigue siendo, dentro de todo el modelo de control de acceso que definimos en la sección anterior, el punto donde más depende de las personas y no del sistema: por más que validemos roles en el backend, si una contraseña se filtra o se adivina, el atacante entra con todos los

**ANEP****UTU**DIRECCIÓN GENERAL
DE EDUCACIÓN
TÉCNICO PROFESIONALInstituto Tecnológico Superior
UTU

permisos de esa cuenta. Por eso no nos quedamos solo con “usar contraseñas fuertes” como indicación genérica, sino que definimos una política concreta para Canopus, diferenciada según el impacto que tendría comprometer cada rol.

Diseñamos esta política priorizando contraseñas largas y memorables por sobre reglas de complejidad forzada, porque en la práctica ese tipo de reglas termina generando contraseñas que se anotan en un papel, se reutilizan entre distintos servicios o se resuelven con un patrón predecible (como cambiar una “o” por un “0”). Preferimos apoyarnos en la longitud, que es el factor que más dificulta un ataque de fuerza bruta, y reservar la exigencia más estricta (mayor longitud y autenticación multifactor obligatoria) para los roles que concentran más permisos, en vez de aplicar la misma regla pareja a todos los usuarios:

Criterio	Requisito
Longitud mínima	10 caracteres para el rol Participante y 14 caracteres para Organizador de torneo y Administrador general, ya que estos dos últimos concentran permisos para modificar resultados y configuración. Recomendamos usar frases de contraseña largas y fáciles de recordar, por ejemplo “Canopus2026–TorneoFutbol!”.
Complejidad	Priorizamos la longitud por sobre la complejidad forzada: recomendamos combinar mayúsculas, minúsculas, números y algún símbolo, integrados naturalmente en una frase memorable y no como sustituciones predecibles (evitamos patrones como “P@ssw0rd”). No se permite usar datos fáciles de adivinar (nombre propio, nombre de equipo público, fecha de nacimiento o nombre del instituto) como parte de la contraseña.
Contraseñas por defecto	Las cuentas que crea un administrador nunca quedan con una contraseña genérica o predecible (por ejemplo “torneo123”); el sistema va a forzar el cambio de contraseña en el primer inicio de sesión de cada usuario nuevo.
Intentos fallidos	Después de 5 intentos fallidos consecutivos, la cuenta se bloquea temporalmente y notificamos al usuario, para dificultar la fuerza bruta sin afectar de forma permanente a alguien que simplemente se equivocó.
Almacenamiento	Las contraseñas nunca se guardan en texto plano en la base de datos; se almacenan siempre como hash mediante password_hash() de PHP (bcrypt),

**ANEP****UTU**DIRECCIÓN GENERAL
DE EDUCACIÓN
TÉCNICO PROFESIONALInstituto Tecnológico Superior
UTU

Criterio	Requisito
	de modo que, aunque la base se filtrara, las contraseñas originales no quedan expuestas.
Cuentas compartidas	Queda prohibido compartir una misma cuenta entre varios usuarios, incluso entre organizadores de un mismo torneo, porque impide saber quién hizo una acción y rompe la trazabilidad de la auditoría.
No reutilización	El sistema no permite reutilizar las últimas 3 contraseñas de una misma cuenta, y recomendamos a los usuarios no repetir la misma contraseña que usan en otras plataformas.
Autenticación multifactor	Obligatoria para el rol Administrador general y recomendada para Organizador de torneo, por ser los roles cuyo compromiso tendría mayor impacto sobre el sistema.
Recuperación de contraseña	Se realiza únicamente mediante un enlace temporal de un solo uso enviado al correo registrado. Nunca enviamos la contraseña actual por correo ni permitimos restablecerla con preguntas fáciles de adivinar, y todo cambio queda registrado en el log de auditoría (usuario, fecha e IP) sin guardar en ningún caso el valor de la contraseña.

En conjunto, estas reglas buscan un equilibrio entre seguridad real y usabilidad: no le pedimos a un participante que solo consulta sus resultados el mismo nivel de exigencia que a un administrador que puede crear usuarios o borrar torneos, pero tampoco dejamos ningún rol sin protección mínima. Vamos a documentar esta misma política en el manual de usuario, para que quede claro desde el primer inicio de sesión por qué el sistema exige lo que exige.

6. Políticas y medidas de seguridad generales

Estas políticas resumen y conectan todo lo relevado en las secciones anteriores: se desprenden directamente de las amenazas identificadas, de las vulnerabilidades de infraestructura y del modelo de control de acceso y contraseñas. Las detallamos una por una porque cada una responde a un problema concreto que ya explicamos, y no queremos que queden como una lista genérica de buenas prácticas:

**ANEP****UTU**DIRECCIÓN GENERAL
DE EDUCACIÓN
TÉCNICO PROFESIONALInstituto Tecnológico Superior
UTU

Política	En qué consiste y cómo la aplicamos
Principio de mínimo privilegio	Tanto las cuentas del servidor (root, admin-canopus, apache, mysql, deploy-canopus, como los roles de la aplicación van a tener únicamente los permisos necesarios para su función. Lo vamos a revisar como antes de cada entrega: si una cuenta o un rol tiene un permiso que no usa, se lo sacamos.
Autenticación multifactor (MFA)	Obligatoria para Administrador general y recomendada para Organizador de torneo, porque son los roles cuyo compromiso tendría mayor impacto. La vamos a implementar con una aplicación de autenticación (TOTP), pidiendo el código además de la contraseña en el login de esos dos roles.
Validación de datos en frontend y backend	La validación en el frontend (JavaScript) es solo para mejorar la experiencia del usuario y evitarle viajes innecesarios al servidor, pero la validación real (la que decide si un dato entra o no a la base) siempre va a estar en el backend con PHP, para prevenir inyección SQL y XSS aunque alguien desactive o esquive el JavaScript del navegador. De la misma forma, el rol del usuario se valida siempre en el backend, nunca solo ocultando opciones en la interfaz.
Uso obligatorio de HTTPS	Toda comunicación entre el navegador y el servidor va a viajar cifrada mediante certificado SSL/TLS, incluyendo el panel de administrador, el de organizador y el módulo de consulta pública, para que ni las credenciales ni los resultados puedan interceptarse en tránsito .
Firewall con reglas mínimas necesarias	Vamos a dejar cerrado por defecto todo puerto que Canopus no use, y a abrir únicamente los que la aplicación realmente necesita (HTTPS hacia afuera, y los puertos internos de Docker entre contenedores). Cualquier puerto nuevo que se necesite en el futuro se va a abrir de forma explícita y documentada, no dejarlo abierto “por las dudas”.
Actualización periódica	Del sistema operativo y de los paquetes utilizados (Apache, PHP, MySQL, Docker), con una revisión fija antes de cada entrega del proyecto, para reducir la ventana de tiempo en la que una vulnerabilidad ya conocida y corregida sigue presente en nuestro servidor.
Respallos automatizados	De la base de datos, guardados fuera del servidor de producción y no en el mismo disco que la aplicación, con pruebas periódicas de restauración (no alcanza con generar el respaldo si nunca comprobamos que se puede recuperar). Esto también nos protege ante cualquier pérdida accidental de datos, no solo ante un ataque.
Registro y auditoría	De las acciones administrativas y de los organizadores, conforme a lo

**ANEP****UTU**DIRECCIÓN GENERAL
DE EDUCACIÓN
TÉCNICO PROFESIONALInstituto Tecnológico Superior
UTU

Política	En qué consiste y cómo la aplicamos
	definido en la sección 4: quién hizo el cambio, cuándo y desde dónde, para poder investigar cualquier incidente y para que cada usuario sepa que sus acciones quedan registradas.
Concientización del equipo	Sobre phishing e ingeniería social, ya que varias de las amenazas identificadas en la sección 2 dependen del factor humano y no de una falla técnica. Vamos a dejar un procedimiento único y obligatorio para confirmar identidad antes de hacer cambios sensibles a pedido de otra persona (por ejemplo, restablecer una contraseña o modificar un resultado), en vez de que cada integrante decida caso a caso.
Separación de cuentas administrativas y de servicio	En el servidor, conforme a lo ya definido en el Estudio de sistemas operativos cliente y servidor : las cuentas que usamos para administrar el servidor nunca son las mismas que usan los servicios (Apache, MySQL, Docker) para funcionar.

Estas medidas las vamos a ir incorporando de forma progresiva junto con el desarrollo del sistema, priorizando las que dependen del propio código de la aplicación por sobre las que dependen de la infraestructura del servidor: la validación de datos y el hash de contraseñas ya se contemplan desde el diseño del modelo de datos y del backend en PHP, mientras que HTTPS, firewall, MFA y respaldos automatizados quedan planificados para las etapas de despliegue con Docker que corresponden a las próximas entregas del proyecto (Administración de sistemas operativos y Programación Full Stack).

7. Implementaciones de seguridad en el servidor

Para esta parte del trabajo instalamos dos herramientas en la VM de Proxmox para detectar y prevenir intrusos: fail2ban, que vimos en clase con el profe, y OSSEC, que instalamos siguiendo el tutorial de la página oficial.

7.1. Fail2ban

**ANEP****UTU**DIRECCIÓN GENERAL
DE EDUCACIÓN
TÉCNICO PROFESIONALInstituto Tecnológico Superior
UTU

7.1.1. ¿Qué es?

Fail2ban es una herramienta que analiza los logs del sistema (por ejemplo, los intentos de inicio de sesión por SSH) y detecta cuando una IP intenta varias veces con credenciales incorrectas. Si se pasa de una cantidad de intentos fallidos, esa IP queda baneada automáticamente por un tiempo determinado, usando el firewall. Básicamente sirve para frenar ataques de fuerza bruta.

7.1.2. Instalación

Primero lo instalamos con el gestor de paquetes dnf:

```
bash
sudo dnf install fail2ban
```

Después probamos que la configuración esté bien con el modo test:

```
bash
sudo fail2ban-client -t
```

Y chequeamos que haya quedado instalado viendo la versión:

```
bash
fail2ban-client version # para verificar si se instaló
```

Configuración del jail de SSH

Fail2ban trabaja con archivos de configuración llamados "jails", donde le decimos qué servicio queremos que monitoree y con qué reglas banea. Creamos el archivo:

```
bash
```



ANEP



UTU

DIRECCIÓN GENERAL
DE EDUCACIÓN
TÉCNICO PROFESIONAL



Instituto Tecnológico Superior
UTU

```
sudo micro /etc/fail2ban/jail.d/sshd.local
```

Y adentro pusimos:

```
ini
[sshd]
enabled = true
port = ssh
backend = systemd
maxretry = 3
findtime = 10m
bantime = 2h
banaction = firewallcmd-rich-rules
```

Y por último:

```
sudo systemctl start fail2ban
sudo systemctl enable fail2ban
sudo systemctl status fail2ban
sudo fail2ban-client status sshd
```

Con esto confirmamos que el servicio estaba corriendo y el jail de sshd activo.

7.2 OSSEC

7.2.1. ¿Qué es?

OSSEC es un HIDS (Host-based Intrusion Detection System), o sea un sistema de detección de intrusos basado en host. A diferencia de fail2ban

**ANEP****UTU**DIRECCIÓN GENERAL
DE EDUCACIÓN
TÉCNICO PROFESIONALInstituto Tecnológico Superior
UTU

(que solo banea por intentos de login), OSSEC monitorea todo el sistema: revisa logs, chequea la integridad de archivos (si algo se modificó sin autorización), detecta rootkits y puede mandar alertas por mail cuando pasa algo raro.

7.2.2. Instalación

Primero agregamos el repositorio de Atomicorp, que es el que provee los paquetes de OSSEC, corriendo el instalador oficial:

```
bash
```

```
wget -O - https://updates.atomicorp.com/installers/oum | bash
```

le sacamos el `-q` porque si hace algo, con el `-q` no nos mostraria a pesar de hace algo

Una vez agregado el repositorio, instalamos el paquete de OSSEC

```
bash
```

```
sudo dnf install ossec-hids-server
```

Durante la instalación el sistema nos pidió confirmar algunas configuraciones iniciales (como si queríamos notificaciones por correo y el chequeo de integridad de archivos con syscheck), y seguimos las opciones por defecto que sugiere el instalador.

Puesta en marcha y verificación

Una vez instalado, iniciamos el servicio:

```
bash
```

```
sudo /var/ossec/bin/ossec-control start
```

**ANEP****UTU**DIRECCIÓN GENERAL
DE EDUCACIÓN
TÉCNICO PROFESIONALInstituto Tecnológico Superior
UTU

Y verificamos que estuviera corriendo:

```
bash
```

```
ps aux | grep ossec
```

La configuración general queda en `/var/ossec/etc/ossec.conf`, donde después se pueden ajustar las reglas de monitoreo

7.3 Grafana

7.3.1 ¿Qué es?

Grafana es una plataforma de código abierto para la visualización de datos, monitoreo y análisis de sistemas, que permite convertir métricas complejas en paneles y gráficos interactivos.

Sirve para:

Unificar fuentes de datos: Se conecta a múltiples bases de datos y herramientas (como Prometheus, InfluxDB, Elasticsearch, MySQL, entre otras) sin necesidad de migrar o cambiar la información de lugar.

Crear paneles (Dashboards): Permite diseñar tableros visuales personalizados con gráficos de líneas, barras, mapas de calor e histogramas para ver el estado de servidores, aplicaciones o redes en tiempo real.

Configurar alertas: Avisa de forma automática cuando una métrica supera un límite establecido (por ejemplo, si el uso de memoria de un servidor pasa del 90%).

7.3.2 Instalación

Seguimos los pasos de la página de Grafana para la instalación



ANEP



UTU

DIRECCIÓN GENERAL
DE EDUCACIÓN
TÉCNICO PROFESIONAL



Instituto Tecnológico Superior
UTU

```
wget -O gpg.key https://rpm.grafana.com/gpg.key
```

```
sudo rpm --import gpg.key
```

Create /etc/yum.repos.d/grafana.repo with the following content:

```
[grafana]
```

```
name=grafana
```

```
baseurl=https://rpm.grafana.com
```

```
repo_gpgcheck=1
```

```
enabled=1
```

```
gpgcheck=1
```

```
gpgkey=https://rpm.grafana.com/gpg.key
```

```
sslverify=1
```

To install Grafana Enterprise, run the following command:

```
sudo dnf install grafana-enterprise
```

luego para que iniciara seguimos los siguientes pasos.

```
1.sudo systemctl daemon-reload
```

```
sudo systemctl start grafana-server
```

```
2.sudo systemctl status grafana-server
```

```
3.sudo systemctl enable grafana-server.service
```

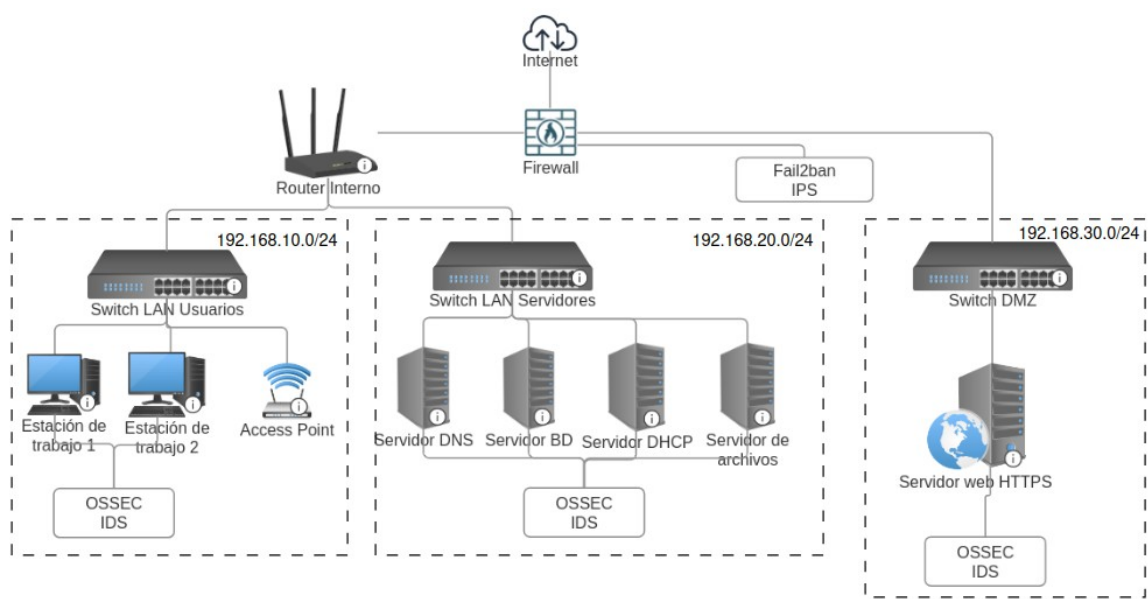
**ANEP****UTU**DIRECCIÓN GENERAL
DE EDUCACIÓN
TÉCNICO PROFESIONALInstituto Tecnológico Superior
UTU

8. Esquema de red

9. Encriptación de contraseñas

El metodo utilizado para la encriptación de contraseñas es el metodo PASSWORD_BCRYPT, directamente guardado en la base de datos en la clase de controlador de Usuario:

```
public function registrar(string $nombre, string $apellido, string $email,
string $password, int $id_rol): void {
    if (!empty($nombre) && !empty($apellido) && filter_var($email,
FILTER_VALIDATE_EMAIL) && !empty($password) && $id_rol > 0) {
        $password_hash = password_hash($password,
PASSWORD_BCRYPT);
        $nuevoUsuario = new Usuario(null, $id_rol, $email,
$password_hash, $nombre, $apellido);
        $this->repository->guardar($nuevoUsuario);
        header('Location: index.php?action=listar');
    } else {
        echo "Datos inválidos.";
    }
}
```

**ANEP****UTU**DIRECCIÓN GENERAL
DE EDUCACIÓN
TÉCNICO PROFESIONALInstituto Tecnológico Superior
UTU

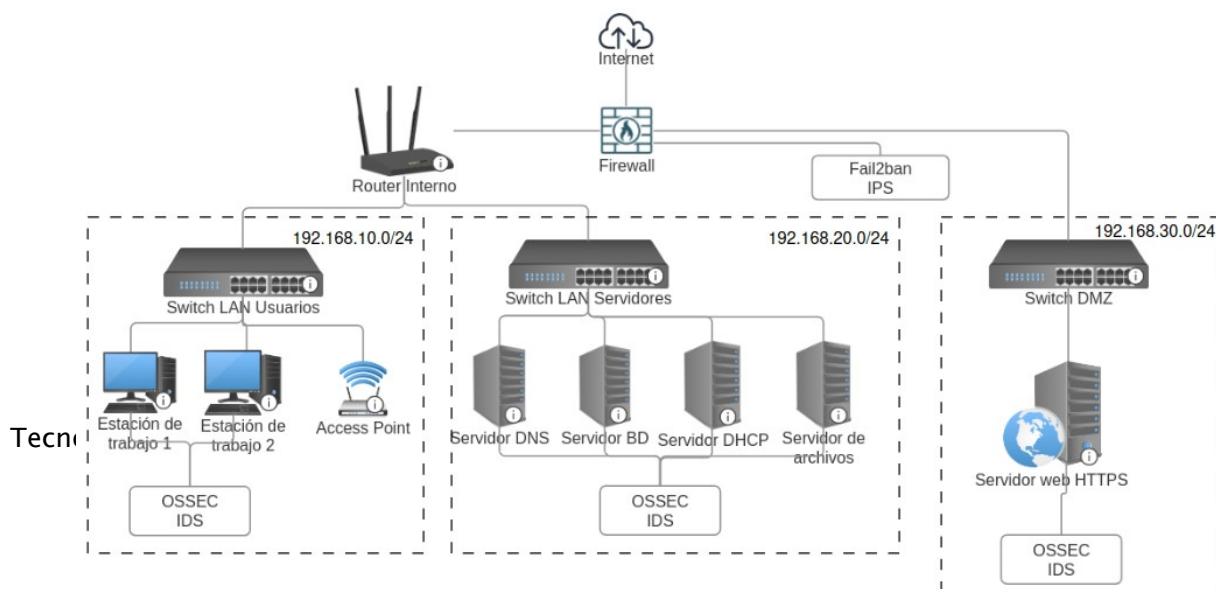
10. Bibliografía

ANEP-UTU, Instituto Tecnológico Superior Arias-Balparda. (2026). Controles de acceso, roles de usuarios y políticas de contraseñas (partes 1 y 2) [Material de la unidad curricular Ciberseguridad].

ANEP-UTU, Instituto Tecnológico Superior Arias-Balparda. (2026). Ciberseguridad: Tipos de ataque [Material de la unidad curricular Ciberseguridad].

Phishing y MFA (TOTP):

Internet Engineering Task Force. (2011). *RFC 6238: TOTP — Time-Based One-Time Password Algorithm*. IETF. <https://datatracker.ietf.org/doc/html/rfc6238>



**ANEP****UTU**DIRECCIÓN GENERAL
DE EDUCACIÓN
TÉCNICO PROFESIONALInstituto Tecnológico Superior
UTU**Ataques a contraseñas (bloqueo por intentos fallidos):**

OWASP Foundation. (2024). *Authentication Cheat Sheet*. OWASP Cheat Sheet Series.

https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

Explotación de vulnerabilidades (SQL injection y XSS):

OWASP Foundation. (2024). *SQL Injection Prevention Cheat Sheet*. OWASP Cheat Sheet Series.

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

OWASP Foundation. (2024). *Cross Site Scripting Prevention Cheat Sheet*. OWASP Cheat Sheet Series.

https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

MITM (HTTPS/HSTS):

OWASP Foundation. (2024). *Transport Layer Security Cheat Sheet*. OWASP Cheat Sheet Series.

https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Security_Cheat_Sheet.html

Docker Inc. (2026). *Dockerfile reference*. Docker Docs.

<https://docs.docker.com/reference/dockerfile/>

Docker Inc. (2026). *Writing a Dockerfile: Use a least privileged user*. Docker Docs.

<https://docs.docker.com/get-started/docker-concepts/building-images/writing-a-dockerfile/>

Docker Inc. (2026). *Compose Deploy Specification: Resources*. Docker Docs.

<https://docs.docker.com/reference/compose-file/deploy/>

Docker Inc. (2025). *Networking in Compose: Internal networks*. Docker Docs.

<https://docs.docker.com/compose/how-tos/networking/>

Docker Inc. (2024). *Docker Engine security*. Docker Docs.

<https://docs.docker.com/engine/security/>

Inyección SQL con PDO (base de datos):

PHP Group. (2026). *PDOStatement — Prepared statements*. PHP Manual.

<https://www.php.net/manual/en/pdo.prepared-statements.php>

**ANEP****UTU**DIRECCIÓN GENERAL
DE EDUCACIÓN
TÉCNICO PROFESIONALInstituto Tecnológico Superior
UTU

Usuario de MySQL con permisos limitados:
Oracle Corporation. (2026). *GRANT statement*. MySQL 8.4 Reference Manual.
<https://dev.mysql.com/doc/refman/8.4/en/grant.html>

Hash de contraseñas (bcrypt):
PHP Group. (2026). *password_hash*. PHP Manual.
<https://www.php.net/manual/en/function.password-hash.php>
OWASP Foundation. (2024). *Password Storage Cheat Sheet*. OWASP Cheat Sheet Series.
https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Firewall (ufw) y fail2ban:
Canonical Ltd. (2026). *Firewalls — UFW*. Ubuntu Server Documentation.
<https://documentation.ubuntu.com/server/how-to/security/firewalls/>
Fail2ban Developers. (2026). *Fail2ban*. GitHub. <https://github.com/fail2ban/fail2ban>

Sesiones en PHP:
PHP Group. (2026). *Sessions*. PHP Manual.
<https://www.php.net/manual/en/book.session.php>

Longitud sobre complejidad:
National Institute of Standards and Technology. (2025). *SP 800-63B: Digital Identity Guidelines — Authentication and Authenticator Management*. NIST.
<https://csrc.nist.gov/pubs/sp/800/63/b/upd2/final>

OWASP Foundation. (2024). *Forgot Password Cheat Sheet*. OWASP Cheat Sheet Series.
https://cheatsheetseries.owasp.org/cheatsheets/Forgot_Password_Cheat_Sheet.html

Registro y auditoría:
OWASP Foundation. (2024). *Logging Cheat Sheet*. OWASP Cheat Sheet Series.
https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html